

## A b s t r a c t   F a c t o r y

# Abstract Factory Pattern

Families of related objects

## DEFINITION

Provide an interface to create families of related objects without specifying concrete classes.

Abstract Factory raises Factory Method from one product to a set. The factory interface declares a create-method per product (button, checkbox, menu), and each concrete factory returns one mutually-consistent family — all Mac, or all Windows. Clients receive products through the abstract interface and never see, or mix, concrete classes from two families.

## WHY IT MATTERS

When a product set must stay internally consistent — a whole UI theme, a database's connection + command + reader — letting clients new pieces individually risks mixing a Mac button with a Windows checkbox. A single factory that vends the entire family guarantees compatibility and confines every concrete class to one place, so switching themes is one wiring change. The cost: the line-up is fixed — adding a new product kind touches the interface and every factory.



# — How it works

**AbstractFactory** Declares one creation method per product in the family, all returning interfaces.

**ConcreteFactory** Implements them to build one consistent family (MacButton + MacCheckbox).

**Products** The related interfaces (Button, Checkbox) clients use; the concretes stay hidden.

## HOW TO APPLY

1. Identify products that must vary together as a set (a theme, a platform, a driver).
2. Declare an AbstractFactory with one create-method per product, returning interfaces.
3. Implement one ConcreteFactory per family, building only that family's concretes.
4. Inject the chosen factory at the edge; clients build everything through it.

## LITMUS TEST

Do several objects have to come from the same family to be compatible? If swapping one detail forces swapping a matching set, hide the set behind one factory.



## — Smell → fix

The client news platform widgets directly, so families can get mixed:

```
function buildUI(os) {  
  const btn = os === 'mac'  
    ? new MacButton() : new WinButton()  
  const cb = new WinCheckbox() // oops: mixed family  
}
```

↓ VEND THE WHOLE FAMILY FROM ONE FACTORY

```
interface UIFactory {  
  button(): Button; checkbox(): Checkbox  
}  
  
class MacFactory implements UIFactory {  
  button(){ return new MacButton() }  
  checkbox(){ return new MacCheckbox() }  
}
```

buildUI now takes a UIFactory and calls button() / checkbox() — one ConcreteFactory keeps the family consistent. Switching to Windows is one injected object, not scattered news.

### USE WHEN

- ✓ UI toolkits and themes
- ✓ Cross-platform widget sets — swap a whole family at once

### AVOID WHEN

- ▲ Products aren't actually related or co-varying



# — Trade-offs & pitfalls

## PROS

- + Guarantees compatible products
- + Isolates concrete classes

## CONS

- − Adding a new product kind touches every factory

## COMMON PITFALLS

- ▲ Open for new families, closed against new product kinds: adding a third product changes the interface and every factory.
- ▲ Over-applying when products don't actually co-vary — you get factory ceremony around unrelated objects.
- ▲ Letting a concrete product leak to the client (an instance of MacButton) — it re-couples what the factory hid.

## WHERE YOU SEE IT

- ▶ Cross-platform UI toolkits: a Mac vs Windows vs Linux widget family behind one factory.
- ▶ A data-access factory producing a matching Connection + Command + DataReader per database.
- ▶ Look-and-feel / theming systems that swap an entire component set at once.

## SEE ALSO

|                |                                                                                 |
|----------------|---------------------------------------------------------------------------------|
| Factory Method | the per-product building block Abstract Factory is usually composed from        |
| Builder        | both isolate construction — Builder assembles one complex object step by step   |
| Prototype      | a factory can build its family by cloning prototype instances instead of newing |
| Singleton      | a concrete factory is typically kept as a single shared instance                |



## — Interview & sources

### Abstract Factory vs Factory Method?

Factory Method makes one product via subclassing; Abstract Factory makes a family of related products via composition — and is often implemented with several factory methods.

### When does Abstract Factory hurt?

When the line-up changes: adding a new product kind edits the interface and every concrete factory. It is open for families, closed against new products.

### How do you keep families consistent?

Each ConcreteFactory builds only its own family and clients receive interfaces, so a Mac button and a Windows checkbox can never be combined.

### Abstract Factory vs Builder?

Abstract Factory returns ready families immediately (what); Builder constructs one complex object step by step (how).

#### REMEMBER

One factory interface that creates a whole family of related products — swap the family by swapping the factory.

#### SOURCES

GoF — "Design Patterns" (1994): Abstract Factory (creational)

GoF (1994), Related Patterns: Abstract Factory is often built with Factory Method or Prototype; a concrete factory is commonly a Singleton